计算机系统 I

期末复习讲义

两天冲刺版

高频知识点 + 解题模板 + 分章真题

使用说明：

- 本版只保留真题、小测和复习课反复覆盖的内容，不按 PPT 顺序照搬。
- 每章按“**必会知识** → **解题模板** → **易错点** → **真题**”组织，适合两天内完成。
- 标签：**[Exam]** 本课程期末题；**[Quiz]** 本课程小测；**[Recall]** 回忆题；**[Selected]** 补充的典型题。
- 练习不附答案；建议先限时独立完成，再回看对应的解题模板。

涵盖 Lec01–Lec08：信息表示 · 布尔代数 · 组合逻辑 · 运算单元 · 时序逻辑 · ISA · RISC-V · 单/多周期 CPU

目录

1 信息表示 (Information Representation) **3**

2 布尔代数与数字逻辑基础 (Boolean Algebra) **6**

3 组合逻辑设计 (Combinational Logic) **9**

4 算术运算单元 (Arithmetic Unit) **12**

5 时序逻辑 (Sequential Logic) **14**

6 指令集体系结构与寻址方式 (ISA & Addressing) **19**

7 RISC-V 汇编与机器码 (RISC-V Assembly) **21**

8 处理器性能与单周期 CPU (Performance & Single-Cycle) **24**

9 多周期处理器 (Multi-Cycle Processor) **27**

两天复习安排

第 1 天：数字逻辑与时序逻辑

1. 上午：第 1–2 章（信息表示、布尔代数），完成编码/解码和 K 图题。
2. 下午：第 3–4 章（组合逻辑、运算单元），重点做 MUX/译码器、溢出与 Booth。
3. 晚上：第 5 章（时序逻辑），独立完成一道 FSM 设计题和一道时序计算题。

第 2 天：ISA、RISC-V 与 CPU

1. 上午：第 6–7 章，默写 R/I/S/B/U/J 格式，练机器码与汇编互译。
2. 下午：第 8–9 章，沿数据流复习控制信号，完成性能与 CPI 计算。
3. 晚上：重做标记错题，只回看本讲义中对应的“易错点”，不再扩展新内容。

1 信息表示 (Information Representation)

考试知识点与解题方法

数制与转换 (Number Systems & Radix Conversion)

- **R 进制 → 十进制 (radix-r to decimal)**：按权展开 (weighted sum) $\sum A_i r^i$。
- **十进制 → R 进制**：整数部分"除基取余、逆序 (divide by radix, take remainders)"；小数部分"乘基取整、顺序 (multiply by radix, take integer parts)"。
- **二进制 ↔ 八/十六进制**：以小数点为界，每 3 位（octal）/ 4 位（hex）一组，不足补 0。

整数编码 (Integer Encodings)：原 / 反 / 补 / 移码

设 n 位带符号整数，最高位为符号位 (sign bit，0 正 1 负)。

- **原码 (sign-magnitude)**：符号位 + 绝对值，有 $+0/-0$。
- **反码 (one's complement, 1's)**：正数不变；负数除符号位外逐位取反 (bitwise NOT)。$= (r^n - 1) - N$。
- **补码 (two's complement, 2's)**：负数 = 反码 $+1$；$= r^n - N$。唯一的 0；范围 (range) $-2^{n-1} \le X \le 2^{n-1} - 1$（负数比正数多一个）。
- **快捷求补**：从最低位扫描，保留末尾的 0 与第一个 1，其后各位全部取反。恒等式 $\sim x + 1 = -x$。
- **移码 (biased / excess)**：真值 + 偏置常数 (bias)，等于补码符号位取反，常用于浮点阶码 (exponent)。
- **符号扩展 (sign extension)**：无符号补 0，有符号补符号位；**截断 (truncation)** 相当于对 2^k 取模。

IEEE 754 浮点数 (Floating Point)

$V = (-1)^s \times M \times 2^E$，存储为 `[s | exp | frac]`（sign / exponent / fraction）。

- **单精度 (single)** 32 位 = 1+8+23，Bias = 127；**双精度 (double)** 64 位 = 1+11+52，Bias = 1023。
- **规格化 (normalized)**（exp 不全 0 也不全 1）：$E = \exp - \text{Bias}$，$M = 1.\text{frac}$（隐含 1，implied leading 1）。
- **非规格化 (denormalized)**（exp 全 0）：$E = 1 - \text{Bias}$，$M = 0.\text{frac}$；表示 0 及接近 0 的数。
- **特殊值 (special values)**（exp 全 1）：frac 全 0 为 $\pm\infty$ (infinity)；frac 非 0 为 NaN。
- 默认舍入 (rounding)：**向偶数舍入 (round to nearest even)**（恰好一半时使最低有效位为 0）。

- 浮点加法**不满足结合律 (not associative)** $(x+y)+z \neq x+(y+z)$；相等判定危险，应用 ϵ。

非数值数据与存储 (Non-numeric Data & Storage)

- **BCD**：8421 有权码 (weighted code)；十进制每位用 4 位二进制表示（如 $34 \to 0011\,0100$）。余 3 码 (excess-3)、格雷码 (Gray code) 为无权码。
- **ASCII**：7 位；大小写字母相差第 6 位。Unicode 16 位。
- **字节序 (endianness)**：大端 (big-endian) MSB 放低地址；小端 (little-endian) LSB 放低地址（x86、RISC-V 主流）。
- **单位**：容量按 2^{10}（KiB/MiB/GiB）；频率/速率按 10^3。

信息表示题的固定步骤

1. **先定解释方式与位宽**：同一个 bit pattern 只有在指定 unsigned、two's complement、biased 或 IEEE 754 后才有数值。
2. **补码解码**：MSB= 0 按无符号值读；MSB= 1 可用

$$X = U - 2^n$$

   直接得真值，也可取反加 1 求绝对值。
3. **IEEE 754 编码**：十进制 $\to$ 二进制 $\to$ 规格化为 $1.f \times 2^e$ $\to$ s $\to$ $e+\text{Bias}$ $\to$ 截取/舍入 frac；最后检查位数 $1+8+23=32$。
4. **IEEE 754 解码**：先看 exp 是否全 0/全 1，再按规格化数公式计算；不要默认所有数都有隐含的 1。
5. **大小端**：先把数据按字节拆开，再从低地址到高地址排列；load 的符号扩展发生在取出指定宽度之后。

易错点

- "反码"与"按位取反"不是无条件等价：正数的反码仍是本身。
- 符号扩展只能复制原数的符号位；截断后要按新位宽重新解释。
- BCD 是"每个十进制数字 4 bit"，不是把整个十进制数直接转二进制。

真题巩固练习

真题巩固（无答案）

1. **[Quiz]** Represent decimal 75 as (a) a 7-bit binary number and (b) an 8-bit 8421-BCD code. Then place an even-parity bit at the MSB to form a 9-bit code.

2. **[Selected]** Given the bit pattern `0xAD100002`, interpret the value it represents (show your derivation) as each of: (1) a two's complement integer; (2) an unsigned integer; (3) a biased / excess representation; (4) a single-precision floating-point number.

3. **[Exam]** The BCD representation of decimal 34 is? A. `00110100` B. `0011 0100` C. `100010` D. `0010 0010`

4. **[Exam]** What is the 8-bit two's complement representation of $(-35)_{10}$?

5. **[Exam]** Write the IEEE 754 single-precision binary representation of -17.5625.

6. **[Selected]** For the following nine 32-bit values, sort the labels in **ascending order** (sort only; you need not write out the exact values):

Label	Bit pattern	Interpretation
A	0xF0000000	sign-magnitude
B	0xF0000000	two's complement
C	0xF0000000	biased
D	0xF0000000	IEEE 754 float
E	0xF0000000	one's complement
F	0xF7000000	sign-magnitude
G	0xF7000000	two's complement
H	0xF7000000	biased
I	0xF7000000	IEEE 754 float

7. **[Selected]** Let `x7` = `0x10000000`, and the data at that address is `0x1122334455667788`. Execute

```
lb x6, 0(x7)
sd x6, 8(x7)
```

and give the value stored at `0x10000008` on a **big-endian** machine and on a **little-endian** machine.

8. **[Selected]** Convert each of the following numbers from its given base into the other three bases in the table (decimal / binary / octal / hexadecimal): (1) 369.3125_{10}; (2) 10111101.101_2; (3) $(326.5)_8$; (4) $(D6.A)_{16}$.

2 布尔代数与数字逻辑基础 (Boolean Algebra)

考试知识点与解题方法

逻辑门与运算 (Logic Gates & Operations)

- 基本：AND $X{\cdot}Y$、OR $X{+}Y$、NOT $\overline{X}$。扩展：NAND $\overline{XY}$、NOR $\overline{X+Y}$。
- XOR $X \oplus Y = \overline{X}Y + X\overline{Y}$（不同为 1）；XNOR $\overline{X \oplus Y} = XY + \overline{X}\,\overline{Y}$（相同为 1）。
- n 输入真值表 (truth table) 有 2^n 行；真值表是函数**唯一**表示，表达式 (expression) 不唯一。

关键定理 (Key Theorems)

- 吸收律 (absorption) $X + XY = X$， $X(X+Y) = X$； 合并 (combining) $XY + X\overline{Y} = X$。
- **一致性 (consensus)**：$XY + \overline{X}Z + YZ = XY + \overline{X}Z$（删冗余项 YZ）。
- **德摩根 (De Morgan)**：$\overline{XY} = \overline{X} + \overline{Y}$， $\overline{X+Y} = \overline{X}\,\overline{Y}$。
- **对偶 (dual)**：交换 $+ \leftrightarrow \cdot$、$0 \leftrightarrow 1$；对偶 $\neq$ 函数补 (complement，补还要对每个变量取反)。
- 优先级 (precedence)：括号 > NOT > AND > OR。

标准形式与化简 (Standard Forms & Minimization)

- **最小项 (minterm)** m_j：变量为 0 取反；**最大项 (maxterm)** M_j：变量为 1 取反。$m_j = \overline{M_j}$。
- 规范 SOP (sum of products) = $\sum m$(为 1 的行)；规范 POS (product of sums) = $\prod M$(为 0 的行)。两者唯一。
- **成本 (cost)**：文字成本 (literal cost) L；门输入成本 (gate-input cost) $G = L+$（非单文字项数）；$GN = G+$ 反相器数。
- **卡诺图 (Karnaugh map, K-map)**：圈大小为 2 的幂；每个 1 至少被圈一次；圈尽量大；可跨边界；无关项 (don't-care) 仅在有利时圈。
- **主蕴含项 (prime implicant, PI)**：不可再扩大的圈；**必要主蕴含项 (essential PI, EPI)**：覆盖了别的 PI 没覆盖的最小项。

K 图与化简题的考场流程

1. 按固定变量顺序写索引，K 图行列必须使用 Gray 顺序 $00, 01, 11, 10$。
2. **最小 SOP**：圈 1；**最小 POS**：圈 0。每圈大小为 2^k，可跨边界，可重叠，don't-care 只在能扩大圈时使用。
3. 先选 EPI，再用最少的 PI 覆盖剩余最小项。题目要求"all PI/EPI"时，不能只写最终表达式。
4. **SOP 圈 1 的项**：圈内恒 1 的变量写原变量，恒 0 写反变量；**POS 圈 0 的项**：恒 0 写原变量，恒 1 写反变量。

5. 化为 NAND-NAND 时先得最小 SOP，再对整体作两次取反；NOR-NOR 同理从 POS 出发。

成本计算与常用化简抓手

- 文字成本 L：表达式中 literal 总数；门输入成本 G：各 AND/OR 门输入数之和；G_N 再加上反相器输入。计算前先看题目是否把 NOT 单独计数。
- 优先寻找：$X + XY = X$，$XY + X\bar{Y} = X$，$XY + \bar{X}Z + YZ = XY + \bar{X}Z$，以及德摩根变换。
- XOR 适用于“不同”、奇偶校验与可控取反；$X \oplus 0 = X$，$X \oplus 1 = \bar{X}$。

易错点

最小项 m_i 中“该行变量为 0”要写反变量；最大项 M_i 则相反。对偶式只交换 $+ \leftrightarrow \cdot$ 与 $0 \leftrightarrow 1$，函数补还需要对变量取反，两者不是一回事。

真题巩固练习

真题巩固（无答案）

1. **[Quiz]** For the original expression

$$F = A(B + \overline{C}) + \overline{A}(\overline{B} + C) + BCD + \overline{B}\,\overline{C}D,$$

calculate the gate-input cost, simplify F, and calculate the gate-input cost of the simplified expression. State clearly whether inverter inputs are included in your cost model.

2. **[Quiz]** For

$$F(A, B, C, D) = \sum m(1, 3, 7, 11, 15) + \sum d(0, 2, 5),$$

(a) draw the K-map; (b) list every prime implicant and every essential prime implicant; (c) obtain minimum SOP and POS expressions separately.

3. **[Exam]** Simplify $F = A\overline{C} + ABC + AC\overline{D} + CD$. The result equals:
A. $A + CD$　B. $A\overline{C} + AC + CD$　C. $A + AC\overline{D} + CD$　D. $A\overline{D} + CD$

4. **[Exam]** Gate-cost model: each NOT counts as 1, each n-input gate counts as n. Find the total gate cost of the direct implementation of $F = AB\overline{D} + B + C(B + \overline{A}D)$.
A. 10　B. 12　C. 14　D. 16

5. **[Exam]** A 4-variable truth table is given below (X = don't-care).

A	B	C	D	Y	A	B	C	D	Y
0	0	0	0	0	1	0	0	0	X
0	0	0	1	1	1	0	0	1	0
0	0	1	0	X	1	0	1	0	0
0	0	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	0	0	1
0	1	0	1	X	1	1	0	1	1
0	1	1	0	0	1	1	1	0	X
0	1	1	1	0	1	1	1	1	1

(a) Write $Y = \sum m(\cdots) + \sum d(\cdots)$; (b) draw the K-map, find all PIs and EPIs, and write the minimal SOP; (c) use the K-map to find the minimal POS.

6. **[Selected]** Use a K-map to simplify $F = \sum m(0,1,5,7,8,11,13) + \sum d(3,9,12,15)$ into the minimal **NAND-NAND** form (mark the largest groups and their product terms).

7. **[Selected]** How many discrete values can a binary logic variable take? A. 1 B. 2 C. 3 D. 4.

How many output states does a three-state (tri-state) buffer have? When several tri-state buffer outputs are wired together, at most how many may be enabled at once?

3 组合逻辑设计 (Combinational Logic)

考试知识点与解题方法

组合逻辑与设计流程 (Combinational Logic & Design Flow)

- 组合逻辑**无记忆 (memoryless)**：输出只依赖当前输入；无反馈环 (no feedback loop)、一个节点只一个驱动源。
- 设计流程：Specification → Formulation（真值表/方程）→ Optimization → Technology Mapping → Verification。

常用功能块 (Functional Blocks)

- **译码器 (decoder)** $n \to 2^n$：每个输出是一个最小项。n-to-2^n 译码器 + OR 门可实现任意函数（把为 1 行的输出 OR 起来）。带使能 (enable) 可作解复用器 (demultiplexer)。
- **编码器 (encoder)** $2^n \to n$；普通编码器多输入为 1 时输出歧义，需**优先编码器 (priority encoder)**（含 valid 有效位 V）。
- **多路选择器 (multiplexer, MUX)** 2^n-to-1：n 条选择线 (select lines) 选 1 路。用 2^n-to-1 MUX 实现 n 变量函数：输入接选择线，数据端按真值表置 0/1。
- **半加器 (half adder)** $S = X \oplus Y,\ C = XY$；**全加器 (full adder)** $S = X \oplus Y \oplus Z,\ C = XY + XZ + YZ$。
- 七段译码器 (seven-segment decoder)：共阳低有效，共阴高有效。

时序分析与 Verilog (Timing & Verilog)

- 传播延迟 (propagation delay) T_{pd}（最长路径，关键路径 critical path）；污染延迟 (contamination delay) T_{cd}（最短路径）。
- 竞争冒险 / 毛刺 (hazard / glitch)：同一输入经不同延迟路径到达输出产生短暂错误。
- Verilog：`module/endmodule`、`assign`（数据流 dataflow）、`always`（行为 behavioral）；`wire` 接线、`reg` 过程变量；可综合 (synthesizable) vs 仅仿真（testbench 延时/显示）。

组合逻辑设计题模板

1. 读规格，列输入/输出与有效电平；建真值表或直接写规范 SOP/POS。
2. 用 K 图/代数化简，再根据题目限制做 technology mapping（指定 MUX、decoder、NAND 等）。
3. **MUX 实现**：选 k 个变量作 select，对每个 select 组合分别求输出关于剩余变量的子函数，数据端只能接 $0/1/X/\bar{X}$ 或外部门产生的函数。
4. **decoder 实现**：$n \to 2^n$ decoder 的每个输出对应一个 minterm；将 $F = 1$ 的输出 OR 起来。

扩展 decoder 时由高位变量控制 enable。

5. 验证时至少检查全 0、全 1、边界组合与 don't-care 的处理。

延迟、冒险与 Verilog 必会点

- T_{pd} 取任一输入到输出的最长路径；T_{cd} 取最短路径。同一信号沿不同延迟路径汇合可产生 glitch。
- 简单组合赋值用 `assign y = s ? d1 : d0;` 或 `always @*`；组合 `always` 必须覆盖所有分支，否则可能推断出 latch。
- `wire` 表示连线；在过程块中赋值的信号使用 `reg/logic`。题目问"等价三目运算符"时要从外层 select 逐层对照。

易错点

MUX 的数据端编号与 S_1S_0 的位次必须先确认；译码器的 enable 和输出若为低有效，全部逻辑都要同步取反。

真题巩固练习

真题巩固（无答案）

1. **[Exam]** Express the following using the Verilog conditional operator (`?:`); choose the equivalent form:

$$O = \overline{S_0}\,\overline{S_1} I_3 + \overline{S_0} S_1 I_2 + S_0 \overline{S_1} I_1 + S_0 S_1 I_0$$

A. `O = S0 ? (S1 ? I0 : I1) : (S1 ? I2 : I3);`

B. `O = S1 ? (S0 ? I0 : I2) : (S0 ? I1 : I3);`

C. `O = S0 & S1 ? I0 : I3;`

D. `O = S0 ? I0 : I1;`

2. **[Exam/Extra]** Using one 4-to-1 MUX (A, B on the select lines) plus a few external gates, implement

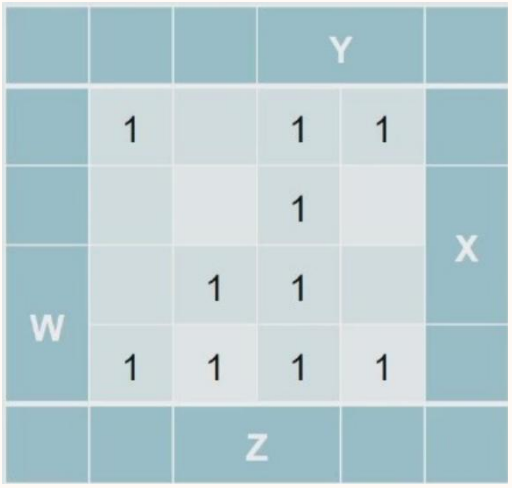
$$F(A,B,C,D) = \sum m(1,3,4,11,12,13,14,15)$$

(Hint: for each case $AB = 00, 01, 10, 11$, find F as a function of C, D.)

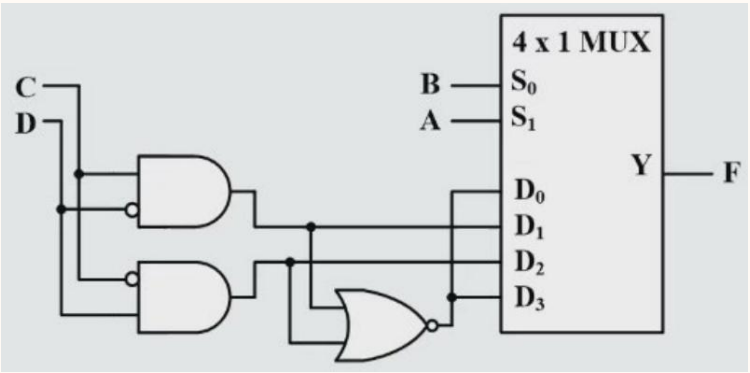
3. **[Exam]** Using only **two 3-to-8 decoders with enable**, **one inverter**, and **OR gates with at most 4 inputs**, implement $F(A,B,C,D) = \sum m(1,3,4,11,12,13,14,15)$. Draw the complete logic diagram.

4. **[Selected]** Design a combinational circuit that compares two 4-bit unsigned numbers A, B and outputs X: $X = 1$ when $A < B$, otherwise $X = 0$.

5. **[Recall]** In the Karnaugh map shown below, how many prime implicants does the function have? A. 4 B. 5 C. 6 D. 7



6. **[Selected]** Analyze the logic circuit below (A, B on select lines S_1, S_0; C, D feed the data inputs $D_0 \sim D_3$ through external gates; output F): build the truth table of F, write the logic expression of F, and state what function this circuit can at most realize.



7. **[Selected]** Design a three-input majority voter with one 3-to-8 decoder and necessary gates. For inputs A, B, C, the output $F = 1$ exactly when at least two inputs are 1. Give the truth table, canonical Boolean function, and complete logic diagram.

4 算术运算单元 (Arithmetic Unit)

考试知识点与解题方法

加法器与减法 (Adders & Subtraction)

- 串行进位加法器 (ripple-carry adder, RCA) $t = N\,t_{FA}$，简单但慢；超前进位 (carry-lookahead, CLA) 用 propagate $P_i = A_i \oplus B_i$、generate $G_i = A_i B_i$ 提前算进位：$C_{i+1} = G_i + P_i C_i$，约 $\log_2 N$ 级的前缀加法器 (prefix adder) 更快。
- 补码统一加减 (add/subtract)：S=1 时把 B 取反、C_0=1，得 $A + \overline{B} + 1 = A - B$。

溢出、进位与标志位 (Overflow, Carry & Flags)

- **无符号 (unsigned) 看进位 carry，带符号 (signed) 看溢出 overflow**（不可混用!）。
- 带符号溢出 (signed overflow)：正 + 正 = 负或负 + 负 = 正；判定 $OF = C_n \oplus C_{n-1}$（不能只看 C_n）。
- 标志位 (flags)：ZF（zero）、SF（sign）、CF（carry/borrow）、OF（signed overflow）。
- 无符号 m 位 × n 位，积 (product) 最多 $m + n$ 位。

乘法 / 除法 / 浮点 (Multiply / Divide / FP)

- 逐次乘法 (sequential multiply) = 移位 (shift) + 加法 (add)；**Booth 算法 (Booth's algorithm)** 把连续 1 压缩：遇 1（左邻 0）减，0（左邻 1）加，中间只移位；阵列乘法器 (array multiplier) 以面积换速度。
- 移位即乘除 2^N：$A \ll N = A \cdot 2^N$，算术右移 (arithmetic shift right) 保留符号位。
- 除法 (division)：恢复除法 (restoring) 试减失败要加回；非恢复除法 (non-restoring) 用下轮加法修正。
- 浮点加法 (FP add)：**对阶 (align exponents) → 加尾数 → 规格化 (normalize) → 检溢出 → 舍入 (round)**；浮点乘法 (FP multiply)：符号异或、阶码相加（减一次 bias）、尾数相乘、规格化、舍入。

加减法、溢出与 Booth 模板

- **统一加减器**：$S = 0$ 时 $B' = B, C_0 = 0$；$S = 1$ 时 $B' = B \oplus 1 = \bar{B}, C_0 = 1$，所以 $A + B' + C_0 = A - B$。
- **无符号数**：加法看 carry-out；减法用补码加法时，carry-out= 1 通常表示无借位。
- **有符号数**：同号相加而结果异号才可能溢出，$OF = C_n \oplus C_{n-1}$。单独的 C_n 不能判断 signed overflow。
- **Booth radix-2**：附加 $Q_{-1} = 0$，每轮看 $Q_0 Q_{-1}$：$01 \to A \leftarrow A + M$，$10 \to A \leftarrow A - M$，

00/11 不变，然后对 A, Q, Q_{-1} 整体算术右移。

- **浮点加法**：对阶 → 尾数加减 → 规格化 → 舍入 → 检查溢出/下溢。浮点乘法则是符号异或、阶码相加减一次 bias、尾数相乘后规格化。

延迟与 ALU 必会关系

- RCA 的进位串行传播，延迟近似随位数线性增长；CLA 使用 $G_i = A_iB_i$、$P_i = A_i \oplus B_i$、$C_{i+1} = G_i + P_iC_i$ 并行生成进位。
- 逻辑左移 k 位相当于乘 2^k（不考虑截断）；算术右移复制符号位，逻辑右移高位补 0。
- m bit 与 n bit 无符号乘法的完整积最多需要 $m + n$ bit。

易错点

答题前先在纸上写“signed 还是 unsigned”。同一组位模式在两种解释下的溢出判据不同，这是本章最常见的失分点。

真题巩固练习

真题巩固（无答案）

1. **[Quiz]** Using one 4-bit full adder and necessary gates, design a controllable 4-bit increment/decrement unit. For control $S = 0$ the circuit increments the 4-bit input by 1; for $S = 1$ it decrements the input by 1. Draw the circuit, label every adder input and carry, and state how overflow/carry should be interpreted.

2. **[Exam]** A combinational multiplier multiplies two 5-bit unsigned numbers. At most how many bits does the product need?
   A. 5 B. 8 C. 9 D. 10

3. **[Exam]** In two's complement arithmetic, when does overflow occur?
   A. adding a positive and a negative number B. carry-out $c_n = 1$ C. the two operands have different sign bits D. $c_n \oplus c_{n-1} = 1$

4. **[Selected]** Use Booth's algorithm (radix-2 or radix-4) to compute $13 \times (-21)$; show the step-by-step process.

5. **[Selected]** Use two's complement subtraction to compute $01010100_2 - 01000011_2$ and $01000011_2 - 01010100_2$; state the carry out of the MSB, whether correction is needed, and the final signed result.

5 时序逻辑 (Sequential Logic)

考试知识点与解题方法

存储元件 (Storage Elements)

- 时序电路 (sequential circuit) = 组合逻辑 + 存储元件；下一状态 (next state) = f(input, current state)。
- **NOR 型 SR 锁存器 (SR latch)** 高有效，禁止输入 (forbidden input) S=R=1；**NAND 型**低有效，禁止输入 S=R=0。
- D 锁存器 (D latch) 电平敏感 (level-sensitive，透明 transparent)；**边沿触发 D 触发器 (edge-triggered D flip-flop)** $Q^+ = D$ 是同步设计标准。
- JK 触发器：11 翻转 (toggle)；T 触发器：1 翻转，$T = Q \oplus Q^+$。
- 主从触发器 (master-slave) 有 ones-catching 问题；建立时间 (setup time) t_s、保持时间 (hold time) t_h。

状态机分析与设计 (FSM Analysis & Design)

- **Moore 机**：输出只依赖现态（状态内标注，状态多更稳定，“Moore is More”）；**Mealy 机**：输出依赖现态 + 输入（边上标输入/输出，状态少）。
- 设计流程：spec → 状态图/表 (state diagram/table) → 状态编码 (state encoding) → 触发器输入方程 (flip-flop input equations) → 输出方程 → 优化 → 映射 → 验证。
- 状态编码位数 $n = \lceil \log_2 m \rceil$；相邻编码可简化卡诺图。
- **时序约束 (timing constraint)**：$t_p \geq t_{pd,FF} + t_{pd,COMB} + t_s$，决定最高频率 (max clock frequency)。

寄存器与计数器 (Registers & Counters)

- 移位寄存器 (shift register)：串入串出 (SISO) / 并行加载 (parallel load)（每级加 MUX 选保持/移位/加载）。
- 同步计数器 (synchronous counter) 共用时钟；波纹计数器 (ripple counter) 仅最低位接时钟，高位被前级触发，延迟 $\approx n\,t_{PHL}$。
- 模 N 计数器 (mod-N counter)：推荐**检测 $N-1$ 后同步清零/加载 (synchronous clear/load)**；**检测 N 后异步清零 (asynchronous clear) 有竞争风险**（suicide counter，勿用）。

触发器特性与激励关系

元件	次态方程	设计时如何求输入
D FF	$Q^+ = D$	$D = Q^+$
T FF	$Q^+ = T \oplus Q$	$T = Q \oplus Q^+$
JK FF	$Q^+ = J\bar{Q} + \bar{K}Q$	$0 \to 1 : J = 1;\ 1 \to 0 : K = 1$

同步时序电路的分析与设计

分析现有电路：

1. 从组合逻辑写各触发器的输入方程（excitation equations）。
2. 代入触发器特性方程得次态 Q^+，另写输出方程。
3. 枚举现态和输入得状态表，再画状态图，最后检查所有未用状态的去向。

从规格设计电路：

1. 画状态图 → 状态表 → 化简等价状态 → 状态编码。
2. 根据状态表求 Q^+ 和输出，再选 D/T/JK 触发器求激励方程。
3. K 图化简，画电路，用正常序列、边界输入和 unused state 验证。

时序约束必会公式

对“触发器 → 组合逻辑 → 触发器”的同步路径：

$$T_{clk} \geq t_{pcq}^{\max} + t_{pd,comb}^{\max} + t_{setup}, \qquad t_{ccq}^{\min} + t_{cd,comb}^{\min} \geq t_{hold}.$$

外部输入到触发器只计算“输入组合路径 $+t_{setup}$”；时钟边沿到外部输出则是“$t_{pcq}+$ 输出组合路径”。最高频率 $f_{max} = 1/T_{clk,min}$。

易错点

- Moore 输出只依赖现态，标在状态内；Mealy 输出依赖现态 + 输入，标在边上，通常状态更少但更易受输入毛刺影响。
- NOR SR latch 的禁止输入为 $S = R = 1$；NAND SR latch 通常为低有效，禁止输入为 00。
- 异步清零不等时钟，可导致竞争；考试设计 mod-N 计数器时优先考虑检测 $N-1$ 后同步装载/清零。

真题巩固练习

真题巩固（无答案）

1. **[Exam]** For an SR latch built from NOR gates, which input drives it into the invalid state?
A. $S{=}0, R{=}0$ B. $S{=}0, R{=}1$ C. $S{=}1, R{=}0$ D. $S{=}1, R{=}1$

2. **[Exam]** Which of the following is correct?
A. A D flip-flop can be built from combinational logic alone.
B. In complex synchronous systems, level-triggered latches usually have better timing robustness than edge-triggered flip-flops.
C. To realize the same function, a Moore machine generally needs more states than a Mealy machine.
D. None of the above.

3. **[Exam]** A traffic-light controller has three states: 00 green, 01 yellow, 11 red; when input $X{=}1$ it stays on yellow. Implement it with two D flip-flops.
(a) Write the complete state-transition table; (b) draw the state diagram; (c) use K-maps to find the next-state equations D_1, D_0; (d) draw the complete sequential circuit.

4. **[Quiz]** Design a controlled 3-bit counter with input C. Starting from 000, when $C = 1$ it repeats

$$000 \to 100 \to 110 \to 111 \to 011 \to 000;$$

when $C = 0$ it repeats

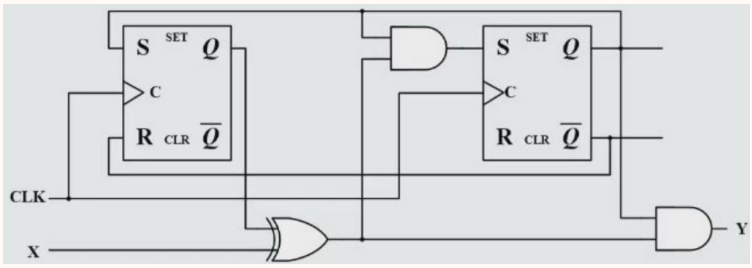
$$000 \to 100 \to 110 \to 010 \to 011 \to 000.$$

Use D flip-flops. Give the state diagram, complete state table (including unused states), simplified next-state equations, output equations, and logic circuit; state how unused states are handled.

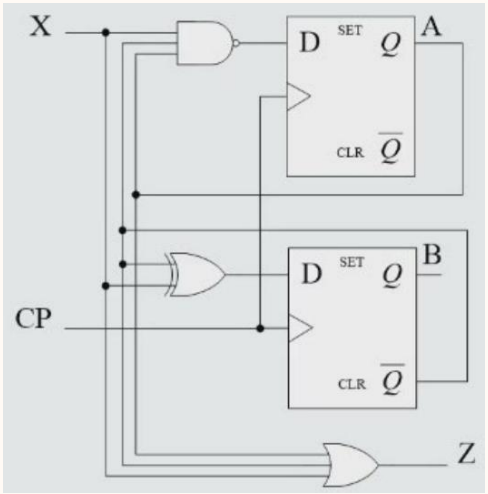
5. **[Selected]** Design a Mealy machine that detects the sequences **101** and **110**: output `10` on detecting 101, `11` on detecting 110, otherwise `00`. Draw the state diagram, state table, next-state equations, output equations, and circuit.

6. **[Selected]** Which function does a basic SR flip-flop **not** provide? A. hold B. toggle C. set D. reset.
Which one has the "ones-catching" problem? A. SR latch B. D latch C. edge-triggered D flip-flop D. SR master-slave flip-flop.

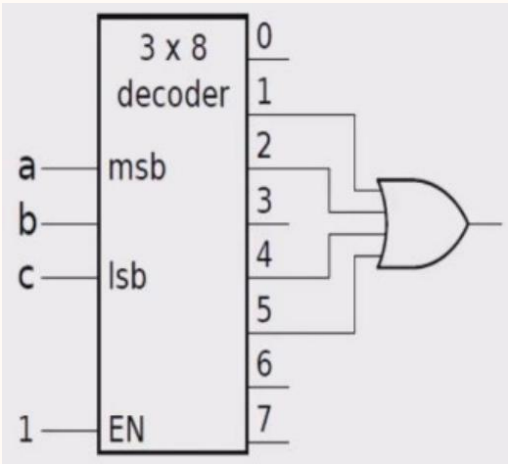
7. **[Recall]** In the sequential circuit below, XOR gate $t_{pd} = 3.0\,\text{ns}$, AND gate $t_{pd} = 1.5\,\text{ns}$, flip-flop $t_{pd} = 3.0\,\text{ns}$, $t_s = 1.0\,\text{ns}$, $t_h = 0.25\,\text{ns}$. How long before the clock rising edge must the external input be stable? A. 8.5 ns B. 7.5 ns C. 6.5 ns D. 5.5 ns



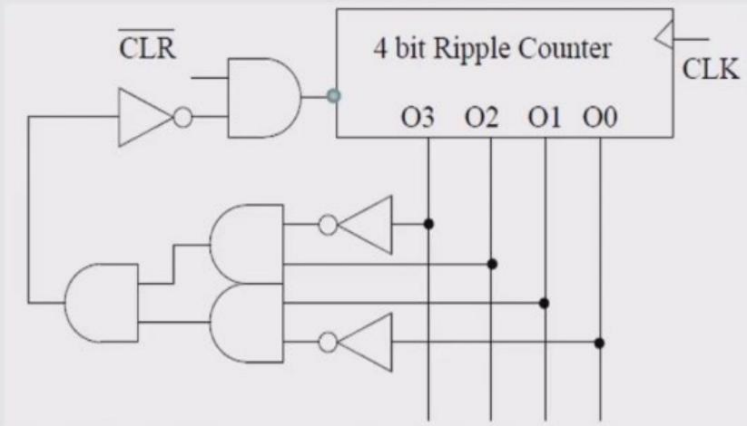
8. **[Recall]** Analyze the circuit below (two D flip-flops, input X, output Z, clock CP): write the output equation and next-state equations, build the state table, and draw the state diagram.



9. **[Recall]** What output function does the circuit below realize? (3×8 decoder, inputs a, b, c, EN=1, outputs combined through an OR gate.)



10. [**Recall**] The circuit below is a mod-N counter with a synchronous reset signal. What is N? A. 6 B. 7 C. 8 D. 9



6 指令集体系结构与寻址方式 (ISA & Addressing)

考试知识点与解题方法

ISA 基础与设计原则 (ISA Basics & Design Principles)

- ISA (instruction set architecture) = 软硬件接口 (hardware/software interface)，定义指令、寄存器、内存模型、异常；同一 ISA 可多种实现。
- 指令 (instruction) = opcode + operands（源/目的/下一条引用）。
- 四原则 (design principles)：规则性利于简单 (simplicity favors regularity)；让常见情况更快 (make the common case fast)；更小更快 (smaller is faster)；好设计需折中 (good design demands compromise)。

操作数个数与寻址方式 (Operands & Addressing Modes)

- 操作数个数 (number of operands)：三地址 (3-address，灵活、字段多)、二地址 (2-address，短、破坏源)、一地址 (1-address，累加器 accumulator)、零地址 (0-address，栈 stack LIFO)。
- **寻址方式 (addressing modes) 与有效地址 (effective address, EA)**：立即 (immediate) $\#K$；直接 (direct) $M[K]$；间接 (indirect) $M[M[K]]$；寄存器 (register) R；寄存器间接 (register indirect) $M[R]$；变址 (indexed) $M[R_m+R_n]$；基址 + 位移 (base + displacement) $M[R_m+X]$；相对 (PC-relative) PC + offset。
- 立即数 $\neq$ 地址；间接比直接多取一层。

CISC vs RISC

- CISC：变长指令 (variable-length)、寻址丰富、微码 (microcode)、语义差距小（x86/VAX）。
- RISC：定长 (fixed-length，如 32 位)、load-store（仅 load/store 访存，ALU 只在寄存器间)、硬布线 (hardwired)、依赖编译器（MIPS/RISC-V）。
- 1980 年后新 ISA 普遍采用 load-store 风格；现代 x86 “CISC 外壳，RISC 内核”。

寻址与指令位宽题

- 先分清“操作数”与“有效地址 EA”：Immediate 直接给操作数；Direct 有 $EA = A$；Register indirect 有 $EA = R$；Base+disp 有 $EA = R_b + d$；PC-relative 有 $EA = PC + d$。
- 间接寻址要多取一层：指令中字段给出的是“存放有效地址的位置”。
- 固定字长指令位数分配：

$$b_{op} = \lceil \log_2 N_{op} \rceil, \quad b_{reg} = \lceil \log_2 N_{reg} \rceil, \quad b_{imm} = b_{inst} - b_{op} - \sum b_{reg}.$$

无符号立即数最大为 $2^{b_{imm}} - 1$；补码范围为 $[-2^{b_{imm}-1}, 2^{b_{imm}-1} - 1]$。

ISA 概念题必会区分

- ISA 定义程序员可见状态及其变化；数据通路、控制器、时钟周期和流水线属于微体系结构实现。同一 ISA 可有多种 CPU 实现。
- 0/1/2/3 地址指令的核心差别是操作数的隐含程度：栈、累加器、寄存器或内存。
- RISC 的考试关键词：fixed length、load-store、regular fields、hardwired control；CISC 的关键词：variable length、rich addressing、microcode。

易错点

“立即寻址”不是“指令里有一个地址”，而是操作数本身就在指令中。若题目问字节偏移，不要再乘 word size；只有题目明确给出 word address 时才做换算。

真题巩固练习

真题巩固（无答案）

1. **[Selected]** The table below lists 9 addressing modes. Write the effective-address E formula for each (Disp = displacement, B = base register, I = index register, S = scale factor, PC = program counter): (1) immediate (2) register (3) direct (4) base (5) base + displacement (6) scaled index + displacement (7) base + index + displacement (8) base + scaled index + displacement (9) PC-relative.

2. **[Selected]** A machine has instruction format: X(2 bits) | OP(6 bits) | source register(3 bits) | destination register(3 bits) | address. There are 8 general-purpose registers (16 bits each) and a main memory of 256K words.
(1) If each memory cell is accessed directly without going through a register and OP = 6 bits, how many bits should the address field be? What should the instruction word length be?
(2) If X=11 designates a general-purpose register as a base register, give a hardware scheme that lets it access every cell of a 1M memory space.

3. **[Exam]** A 32-bit fixed-length ISA: 130 instructions, 64 general-purpose registers; each instruction has an opcode, one destination register *rd*, and one immediate *imm*, in a single word (4 bytes).
(a) Minimum bits for the opcode? (b) Minimum bits for *rd*? (c) Maximum bits for *imm*? (d) If *imm* is an unsigned byte-address offset, what is the largest representable offset? (e) If *imm* is two's complement, what are the maximum and minimum representable values?

4. **[Selected]** Modern computers are built on two key principles: ① instructions are represented as numbers; ② programs can be stored in memory like data; ③ make the common case fast; ④ every instruction can be executed conditionally. The correct ones are: A. ①③ B. ②④ C. ①② D. ③④.

7 RISC-V 汇编与机器码 (RISC-V Assembly)

考试知识点与解题方法

寄存器与指令格式 (Registers & Instruction Formats)

- 32 个寄存器 (registers)：`x0`=0；`x1=ra` 返回地址 (return address)；`x2=sp` 栈指针 (stack pointer)；`a0-a7`(x10-17) 参数/返回值 (arguments/return)；`t*` 临时 (temporary，caller-saved，调用不保留)；`s*` 保存 (saved，callee-saved，调用保留)。
- 六种格式 (formats)(低 7 位 opcode)：**R** (funct7,rs2,rs1,funct3,rd)、**I** (imm[11:0],rs1,funct3,rd)、**S** (store)、**B** (branch)、**U** (lui/auipc)、**J** (jal)。
- 立即数 (immediate) 符号位恒在 inst[31]（符号扩展 sign-extended)；B/J 最低位隐含为 0（2 字节对齐 alignment)。

常用指令与控制流 (Instructions & Control Flow)

- 算术/逻辑 (arithmetic/logic)：`add, addi, sub, and, or, xor`；移位/比较/高位立即数：`sll, srl, sra, slt, sltu, lui, auipc`。
- 访存 (load/store，基址 + 位移 `imm(rs1)`)：`lb/lh/lw`（符号扩展 sign-extend)、`lbu/lhu`（零扩展 zero-extend)、`sb/sh/sw`；RV64 `ld/sd/lwu`。
- 控制流 (control flow，PC 相对)：分支 (branch) `beq/bne/blt/bge/bltu/bgeu`；跳转 (jump) `jal rd,off`（rd=PC+4）、`jalr rd,imm(rs1)`。伪指令 (pseudo-instructions) `j`=`jal x0`，`ret`=`jalr x0,0(x1)`，`nop`=`addi x0,x0,0`。
- 寻址 (addressing)：immediate、register、base + displacement、PC-relative。

RISC-V 格式速查

格式	31–25	24–20	19–15	14–12	11–7	6–0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20,10:1,11,19:12]				rd	opcode

机器码编码/解码模板

1. **解码**：把 32 bit 写成二进制，先取 `inst[6:0]` 找 opcode 和格式，再按固定位置切 rd/rs1/rs2/funct3/funct7，最后重组立即数并符号扩展。
2. **编码**：先根据助记符定格式与 opcode/funct，再把寄存器号写为 5 bit；分支/跳转先算 $target - PC$，检查范围和对齐后再拆立即数位。

3. B/J 偏移的 bit 0 隐含为 0，存入指令的各段位置不连续；“除以 2”与“拆位”不能遗漏。

4. 结果拼好后每 4 bit 转一位十六进制，再用已知字段反向检查一次。

调用约定与常用伪指令

- caller-saved：`t0--t6, a0--a7`；callee-saved：`s0--s11`。非叶子函数若还要调用别的函数，通常需在栈上保存 `ra`。
- `j label=jal x0,label`；`ret=jalr x0,0(ra)`；`mv rd,rs=addi rd,rs,0`；`not rd,rs=xori rd,rs,-1`。
- `lb/lh/lw` 符号扩展，`lbu/lhu` 零扩展；store 只写寄存器低位的指定字节。

易错点

`jalr` 是 I-type，不是 J-type；分支伪指令要注意比较方向和 signed/unsigned，例如 $a \geq b$ 可交换操作数后用 `bge/blt`，不能只按英文名称猜字段。

真题巩固练习

真题巩固（无答案）

1. **[Exam]** In a RISC-V R-type instruction, which field specifies the destination register? A. opcode B. funct3 C. rd D. rs1.

2. **[Exam]** Which format does the `jalr` instruction belong to? A. R B. I C. S D. J (or U).

3. **[Exam]** Memory contents are as follows, with initial `R2 = 0x0004`:

Address	Data
0x1000	0x1008
0x1004	0x100C
0x1008	0x1004
0x100C	0x100D

After executing `lw R1, 0x1004(R2)`, what is the value of `R1`? A. 0x1004 B. 0x1008 C. 0x100C D. 0x100D.

4. **[Selected]** Translate the function `f` into RISC-V assembly (`g` is declared as `int g(int a,int b)`):

```
int f(int a, int b, int c, int d){
  return g(g(a,b), c+d);
}
```

5. **[Selected]** Give the hexadecimal machine code of the following instructions (starting memory address `0x0040 0000`):

```
    ld  x12, 8(x10)
loop:  bne  x12, x13, label
    slli  x13, x12, 4
    jal  x0, loop
label:  jalr  x0, 0(x1)
```

6. **[Quiz]** The same 32-bit pattern `0xFF53AE93` may be interpreted in several ways. Decode it as (a) a 32-bit two's-complement integer, (b) an IEEE 754 single-precision value, and (c) an RV32I instruction. Show all field boundaries and intermediate values.

7. **[Quiz]** The instruction at address `0x00200000` is `Loop: jal x0, L1`; the instruction at `0x00200008` is `beq x3, x4, Loop`; label `L1` is at `0x00200070`. Encode both instructions as 32-bit hexadecimal machine code.

8. **[Quiz]** Convert each pseudoinstruction to the shortest RV32I sequence: (a) `bnez rs, Label`; (b) `j Label`; (c) `mv rd, rs`; (d) `not rd, rs`. Then write the shortest branch sequence to label `IndexOutOfBounds` if $x20 \geq x11$ in signed comparison or if $x20$ is negative.

8 处理器性能与单周期 CPU (Performance & Single-Cycle)

考试知识点与解题方法

性能公式

$$\text{CPU Time} = IC \times CPI \times T_c = \frac{IC \times CPI}{\text{Clock Rate}}$$

- 加权 (weighted) $CPI = \sum CPI_i \times \frac{IC_i}{IC}$（$IC$ = instruction count，CPI = cycles per instruction）。
- “X 比 Y 快 n 倍”$\Leftrightarrow \frac{\text{Time}_Y}{\text{Time}_X} = n$。
- **Amdahl 定律 (Amdahl's law)**：$\text{Speedup} = \frac{1}{(1-F_e) + F_e/S_e}$，上界 $\frac{1}{1-F_e}$；优先优化占比最大部分。
- 三堵墙 (three walls)：功耗墙 (power wall, $P \propto CV^2 f$)、存储墙 (memory wall)、ILP 墙 $\Rightarrow$ 多核 (multicore)。

单周期数据通路与控制 (Single-Cycle Datapath & Control)

- 五阶段 (five stages)：IF → ID → EX → MEM(仅 lw/sw) → WB；指令存储器 (instruction memory) 与数据存储器 (data memory) **分开 (separate)**。
- 主控制信号 (control signals)：ALUSrc、MemtoReg、RegWrite、MemWrite、MemRead、Branch、Jump + ALUOp。
- 两级译码：主译码器 (main decoder, opcode)→ 控制信号 +ALUOp；ALU 译码器 (ALU decoder, ALUOp+funct)→ ALU 控制。
- 时钟周期 (clock cycle) = 关键路径 (critical path)（通常 Load：取指 200+ 读寄存器 100+ALU200+ 读内存 200+ 写寄存器 100 = 800ps）。

单周期控制信号真值表 Control Signal Truth Table (记忆用)

Instr	ALUSrc	MemtoReg	RegWR	MemWR	Branch	Jump	ALUOp
R-type	0	00	1	0	0	0	10
Load (ld)	1	01	1	0	0	0	00
Store (sd)	1	X	0	1	0	0	00
Branch(beq)	0	X	0	0	1	0	01
Jump (jal)	X	10	1	0	0	1	XX
I-ALU(addi)	1	00	1	0	0	0	11

性能计算的统一写法

1. 把题目所有量换成一致单位，先写

$$T_{CPU} = IC \times CPI \times T_c = \frac{IC \times CPI}{f}.$$

2. 有多类指令时，$CPI_{avg} = \sum f_i CPI_i$；若题目给的是指令条数，先用 $f_i = IC_i/IC$。

3. 比较性能使用 $Performance_X/Performance_Y = T_Y/T_X$，不要把“快 n 倍”写成时间乘 n。

4. Amdahl 题先标出可优化比例 F 与局部加速比 S：$Speedup = 1/[(1-F) + F/S]$，并用 $1/(1-F)$ 检查上限。

单周期各类指令的数据流

类型	ALU 做什么	访存/写回	PC 来源
R-type	$rs1$ *op* $rs2$	ALU→ rd	$PC+4$
I-ALU	$rs1$ *op* imm	ALU→ rd	$PC+4$
Load	$rs1 + imm$	Mem→ rd	$PC+4$
Store	$rs1 + imm$	$rs2$ →Mem	$PC+4$
Branch	$rs1 - rs2$ / compare	不写回	$PC + imm$ 或 $PC+4$
JAL	目标 $PC + imm$	$PC+4 \to rd$	跳转目标

数据通路与控制信号题

1. 从语义出发写微操作：读哪个寄存器、ALU 两端选什么、是否读/写数据存储器、写回哪里、PC 如何更新。

2. 画数据路径时先画数据线，多个来源共用同一输入处加 MUX；只在结构确定后命名控制信号。

3. 填控制表时，先填 RegWrite/MemRead/MemWrite 等“会改变状态”的信号，再填 MUX 选择与 ALUOp；无关位写 X。

4. 增加新指令时，按“现有部件能否在一周期内按所需顺序被使用”检查结构冲突，然后列新部件、新 MUX/连线和新控制信号。

易错点

- 单周期 CPU 的每条指令 CPI 都是 1，但时钟周期必须满足最长指令路径，通常由 Load 决定。
- 单周期中指令存储器和数据存储器需同时工作，简单单端口存储器不能直接合并。
- 性能只能用 CPU time 比较；更高时钟频率、更低 CPI 或更少 IC 单独都不足以断定整体更快。

真题巩固练习

真题巩固（无答案）

1. **[Exam]** A single-cycle CPU supports four instruction classes A, B, C, D with frequencies 10%/20%/30%/40% and CPIs 1, 4, 3, ? respectively. After optimization, D's CPI is halved and the overall CPI becomes 3. Find D's original CPI. A. 4 B. 5 C. 6 D. 8.

2. **[Exam]** Which component is **required** in a multi-cycle CPU but **not strictly required** in a single-cycle CPU?
   A. instruction register IR B. program counter PC C. instruction memory D. multiplexer MUX.

3. **[Exam]** Single-cycle CPU execution times by instruction class: R-type 750ps, Load 1000ps, Branch 875ps, I-type 625ps, Jump 600ps, each class occurring with equal frequency.
   (a) Find the clock cycle time and clock frequency; (b) find the average CPI; (c) find the CPU execution time for 10^8 instructions; (d) if a redesign reduces the time in (c) to 80% of the original but increases the total cycle count by 20%, find the new clock frequency.

4. **[Selected]** A function is sped up 20× but accounts for only 40% of total run time; what is the overall speedup?
   If adding an FP unit speeds up FP operations 20× and the overall speedup is 5×, find the FP fraction F_e.

5. **[Selected]** Add a register–memory instruction `Addm rd, rs, rt` (`rd = rs + Mem[rt]`) to the single-cycle datapath.
   (a) Give the binary instruction format; (b) describe the datapath changes needed and draw the datapath for this instruction; (c) list all control signals and their values for this instruction.

6. **[Exam]** A processor with separate instruction/data memories defines three instructions (word = 4 bytes):

Instruction	Semantics
`lw ri, imm`	`Reg[ri]=Mem[imm]`, `PC=PC+4`
`sw ri, imm`	`Mem[imm]=Reg[ri]`, `PC=PC+4`
`add ri, imm`	`Reg[ri]=Reg[ri]+imm`, `PC=PC+4`

   (a) Complete the interface ports of the register file and data memory; (b) draw the complete datapath supporting all three instructions (you may add MUXes, sign extension, etc.); (c) design the control signals for the three instructions and give each signal's value in a table.

7. **[Quiz]** A single-cycle processor runs at 4 GHz and executes 100,000 instructions in 0.03 ms. Find its CPI. A new processor executes instruction classes A, B, C with counts

1500, 2000, 500 and CPIs equal to the previous answer, 1.4, and 2.1, respectively; find its average CPI. Finally, for a multi-cycle RISC-V CPU whose R-type/Load/Store/Branch/Jump instructions require 4/5/4/3/3 cycles and occur with percentages 50%/10%/10%/15%/15%, find the average CPI and total cycle count for 10^6 instructions.

9 多周期处理器 (Multi-Cycle Processor)

考试知识点与解题方法

多周期要点 (Multi-Cycle Essentials)

- 一个时钟周期执行指令的**一个阶段 (one stage)**；时钟周期由**最复杂的单步**决定（而非最长指令）。
- 数据通路变化：**合并存储器 (unified memory)**（指令/数据共用）、**合并 ALU**（去掉额外加法器）、**新增 5 个临时寄存器 (temporary registers)**：IR、MDR、A、B、ALUout。
- 阶段数 (number of stages)：beq/j = 3；R 型/Store = 4；Load = 5。
- 控制信号**随阶段变化** ⇒ 控制器是**时序逻辑 (sequential logic)**（需记当前阶段）。
- 控制器实现：**硬连线 (hardwired)**(FSM+PLA，适合 RISC)vs **微程序 (microprogrammed)**（控制存储器 + 微指令，适合 CISC）。
- 性能：加权 CPI（例 ≈ 4.12），T_c 由存储器读主导（例 325ps）；简单实现下**多周期不一定快于单周期**（时序开销多次支付）；其意义是为流水线 (pipelining) 奠基。

临时寄存器与典型阶段

- **IR**：保存当前指令；**MDR**：保存内存读数据；**A/B**：保存寄存器文件读口输出；**ALUOut**：保存 ALU 本周期结果。
- 典型阶段：R-type = IF–ID–EX–WB；Load = IF–ID–EX–MEM–WB；Store = IF–ID–EX–MEM；Branch/Jump 通常在 3 个左右周期结束（以课件状态表为准）。
- 部件可跨周期复用，因此可合并指令/数据存储器、共用 ALU；代价是需要更多 MUX、临时寄存器和带状态的控制器。

多周期控制与性能题

1. 对每个状态写三件事：本周期读什么、ALU/存储器做什么、时钟边沿写哪个寄存器。
2. 指令类型决定 ID 后的状态转移，同一控制信号在不同状态可取不同值，因此控制器不再是只看 opcode 的纯组合逻辑。
3. 硬布线控制用 FSM/译码逻辑，速度快但修改困难；微程序控制用控制存储器中的微指令，易扩展但速度慢。

4. 性能比较仍用 $T = IC \times CPI \times T_c$；多周期只是缩短 T_c，同时增大 CPI，所以不能凭“时钟更快”判定总体更快。

易错点

单周期的 IF/ID/EX/MEM/WB 是一个周期内的组合数据流；多周期中它们是真正的不同时钟周期。不要把多周期临时寄存器误当作 ISA 可见寄存器。

真题巩固练习

真题巩固（无答案）

1. **[Selected]** List the 5 new temporary registers added in a multi-cycle CPU and the role of each, and state the number of stages and the stage sequence required by each instruction class (beq, R-type, sw, lw).

2. **[Selected]** Given the cycle counts per instruction class in a multi-cycle design: branch/jump 3, R-type/Store 4, Load 5; and the instruction mix Load 25%, Store 10%, Branch 11%, Jump 2%, R-type 52%. Find the weighted average CPI.

3. **[Selected]** Let the multi-cycle design have $T_c = 325$ps, CPI $= 4.12$, and the single-cycle design have $T_c = 925$ps, CPI $= 1$. Find the time to execute 10^{11} instructions for each, and explain why the single-cycle design is actually faster in this example.

4. **[Selected]** Compare a hardwired control unit with a microprogrammed control unit: what are the implementation method, applicable scenario (RISC/CISC), and pros and cons of each?

“Follow the main thread when reviewing: how data is encoded → how logic computes → how state is stored → how instructions are defined → how the processor executes.”